



كلية الهندسة المعلوماتية

جامعة دمشق

السنة الرابعة

قسم الذكاء الصناعي

مشروع المنطق الترجيحي القسم

الوظيفة الاولى

Fuzzy Expert System for Smart Parking Management and Traffic Congestion Reduction

ماسة إياد الزعيم

محمد مجد أحمد سلام المالح

محمد مازن خزاني

محمد غسان فاعل

Smart Fuzzy Parking Management System

Dynamic Pricing + Congestion Control + Parking Time Prediction

1. Project Overview

The Smart Fuzzy Parking Management System is an intelligent application that uses fuzzy logic to optimize urban parking management and reduce traffic congestion. Unlike traditional fixed-price parking systems, this system dynamically adjusts pricing, predicts parking availability wait times, and assesses congestion levels based on real-time traffic conditions.

The system analyzes four input variables and produces three intelligent outputs:

- Dynamic Parking Price — Encourages or discourages parking based on current conditions
- Congestion Control Level — Measures the severity of traffic congestion
- Parking Time Prediction — Estimates the time drivers will wait to find a parking space

By using fuzzy logic, the system mimics human decision-making using linguistic concepts (Low, Medium, High) rather than strict mathematical boundaries, allowing it to adapt naturally to uncertain and continuously changing traffic conditions.

2. Problem Statement

Urban areas face significant traffic congestion partly because drivers spend excessive time searching for available parking spaces. Traditional parking systems:

- Use fixed prices regardless of demand
- Do not respond to real-time conditions
- Cannot adapt to variable traffic patterns
- Fail to optimize space utilization during peak hours

Traffic conditions are inherently uncertain and continuously changing. Fixed decision-making methods cannot provide efficient, adaptive solutions. This project develops a Fuzzy Expert System that responds dynamically to:

- Current parking availability
- Real-time traffic density
- Vehicle arrival rates
- Time-based demand variations

The goal is to reduce unnecessary vehicle movement and traffic congestion by intelligently pricing parking and providing accurate wait-time estimates to drivers.

3. System Architecture

3.1 Input Variables

The system monitors four real-time input parameters, each ranging from 0–100:

Occupancy (%)

Represents the percentage of occupied parking spaces in a specific area.

Fuzzy Set	Definition	Typical Range
Low	Parking area has abundant available spaces	0–40%
Medium	Parking area is partially occupied	30–70%
High	Parking area is almost full	60–100%

Example Values:

- 10% → Low occupancy (plenty of spaces)
- 50% → Medium occupancy (balanced)
- 90% → High occupancy (nearly full)

Traffic Level (%)

Represents the amount of road congestion near the parking area.

Fuzzy Set	Definition	Typical Range
Low	Smooth traffic flow	0–40%
Medium	Normal traffic conditions	30–70%
High	Heavy traffic congestion	60–100%

Example Values:

- 20 → Low traffic (smooth flow)
- 50 → Medium traffic (normal conditions)
- 90 → High traffic (severe congestion)

Arrival Rate (%)

Represents the rate of incoming vehicles searching for parking spaces.

Fuzzy Set	Definition	Typical Range
Low	Few vehicles arriving	0–40%
Medium	Normal vehicle arrival	30–70%
High	Many vehicles arriving	60–100%

Example Values:

- 20 → Slow arrival (few cars searching)
- 50 → Normal arrival (steady stream)
- 90 → Fast arrival (rush of vehicles)

Time Demand (%)

Represents demand variations based on time periods and special events.

Fuzzy Set	Definition	Typical Range
Offpeak	Low demand period (nights, early mornings)	0–40%
Normal	Average demand period (midday, weekdays)	30–70%
Rush	High demand period (peak hours, weekends)	60–100%

Example Values:

- 20 → Offpeak (low demand)
- 50 → Normal (average demand)
- 90 → Rush hour (peak demand)

3.2 Output Variables

The system produces three actionable outputs, each ranging from 0–100:

Dynamic Parking Price (0–100)

A fuzzy price index that dynamically adjusts parking cost based on current conditions.

Price Index	Category	Interpretation	Effect
0–25	Cheap	Very attractive	Encourage parking
25–50	Normal	Standard rate	Balanced incentive
50–75	Expensive	Premium rate	Reduce demand
75–100	Very Expensive	Maximum rate	Strong congestion control

Example Outputs:

- Price index 15 → Cheap (encourage parking, abundant spaces)
- Price index 50 → Normal (balanced conditions)
- Price index 75 → Expensive (reduce demand)
- Price index 92 → Very Expensive (critical congestion control)

Congestion Level (0–100)

Measures the severity of traffic congestion and parking pressure in the area.

Congestion	Status	Traffic Impact	Recommendation
0–40	Low	Smooth flow	Normal operations
40–60	Medium	Moderate delays	Monitor conditions
60–80	High	Significant congestion	Activate pricing controls
80–100	Critical	Severe gridlock	Emergency measures

Parking Time (0–100 → 0–90 minutes)

Predicts the estimated wait time for drivers to find an available parking space. The fuzzy output (0–100) is converted to minutes (0–90 min) using the formula:

$$\text{Parking Time (minutes)} = (\text{Fuzzy Output} / 100) \times 90$$

Time Output	Minutes	Parking Condition	Driver Action
10–15	9–13 min	Very easy	Park immediately
25–35	23–31 min	Easy	Standard search
50–65	45–58 min	Difficult	Consider alternatives
80–100	72–90 min	Very difficult	Seek alternative parking

4. Membership Functions

The system uses trapezoidal and triangular membership functions to represent uncertain parking conditions.

4.1 Input Membership Functions

Low States (Trapezoidal)

`trapmf(RANGE, [0, 0, 25, 40])`

- Values 0–25% have full membership (degree = 1.0)
- Membership decreases linearly from 25–40%
- Represents clearly low conditions over a range

Used for: Low occupancy, Low traffic, Low arrival rate, Offpeak demand

Medium States (Triangular)

`trimf(RANGE, [30, 50, 70])`

- Peak membership at 50% (degree = 1.0)
- Membership decreases toward 30% and 70%
- Represents balanced conditions with a clear center point

Used for: Medium occupancy, Medium traffic, Medium arrival rate, Normal demand

High States (Trapezoidal)

`trapmf(RANGE, [60, 80, 100, 100])`

- Values above 80% have full membership (degree = 1.0)
- Membership decreases linearly from 60–80%
- Represents clearly high conditions across the upper range

Used for: High occupancy, High traffic, High arrival rate, Rush demand

4.2 Output Membership Functions

Price Membership Functions

Cheap (Trapezoidal): `trapmf(RANGE, [0, 0, 20, 40])`

Normal (Triangular): `trimf(RANGE, [30, 50, 70])`

Expensive (Triangular): `trimf(RANGE, [60, 80, 95])`

Very Expensive (Trapezoidal): `trapmf(RANGE, [85, 95, 100, 100])`

Congestion Membership Functions

Low (Trapezoidal): `trapmf(RANGE, [0, 0, 15, 40])`

Medium (Triangular): `trimf(RANGE, [20, 50, 80])`

High (Trapezoidal): `trapmf(RANGE, [60, 80, 100, 100])`

Parking Time Membership Functions

Very Short (Trapezoidal): `trapmf(RANGE, [0, 0, 15, 30])`

Short (Triangular): `trimf(RANGE, [20, 35, 50])`

Long (Triangular): `trimf(RANGE, [45, 65, 80])`

Very Long (Trapezoidal): `trapmf(RANGE, [75, 90, 100, 100])`

4.3 Why These Membership Functions?

Trapezoidal for Low/High States:

- These conditions can remain clearly true over a range (e.g., 85–100% occupancy is unambiguously "full")
- Provides efficient computation
- Easier interpretation for traffic experts

Triangular for Medium/Normal States:

- Represents balanced situations with a clear peak (e.g., 50% = perfectly balanced)
- Provides natural gradation around the center point
- Reflects real-world conditions where extreme balance is uncommon

Overall Benefits:

- Faster computation compared to Gaussian functions
- Clearer expert interpretation
- Easier to tune based on domain knowledge
- Natural linguistic representation

5. Fuzzy Rule Base

The expert knowledge is encoded in 14 intelligent rules that govern system behavior. Rules are grouped by occupancy level for clarity.

5.1 Low Occupancy Rules

Rule 1: Low Occupancy + Low/Free Traffic

IF (Occupancy is Low) *AND* (Traffic is Low OR Arrival Rate is Low) *THEN* Price = Cheap, Congestion = Low, Parking Time = Very Short

- When parking is abundant and traffic is light, encourage parking with low prices.

Rule 2: Low Occupancy + High Traffic

IF (Occupancy is Low) *AND* (Traffic is Medium OR Traffic is High) *THEN* Price = Normal, Congestion = Medium, Parking Time = Short

- Despite available parking, heavy traffic makes parking less attractive. Use normal pricing.

5.2 Medium Occupancy Rules

Rule 3: Medium Occupancy + Low Traffic + Offpeak

IF (Occupancy is Medium) *AND* (Traffic is Low) *AND* (Time Demand is Offpeak) *THEN* Price = Normal, Congestion = Low, Parking Time = Short

- During offpeak periods with light traffic, maintain normal pricing to encourage parking.

Rule 4: Medium Occupancy + Balanced Conditions

IF (Occupancy is Medium) *AND* (Traffic is Medium OR Arrival Rate is Medium OR Time Demand is Normal) *THEN* Price = Normal, Congestion = Medium, Parking Time = Short

- When all factors are balanced, maintain normal pricing and conditions.

Rule 5: Medium Occupancy + High Pressure

IF (Occupancy is Medium) *AND* (Traffic is High OR Arrival Rate is High OR Time Demand is Rush) *THEN* Price = Expensive, Congestion = High, Parking Time = Long

- When any pressure indicator is high, increase prices to manage demand and prevent congestion.

5.3 High Occupancy Rules

Rule 6: High Occupancy + Low Traffic

IF (Occupancy is High) AND (Traffic is Low) *THEN* Price = Expensive, Congestion = Medium, Parking Time = Long

- Even with light traffic, high occupancy requires price increases to free up spaces.

Rule 7: High Occupancy + High Pressure

IF (Occupancy is High) AND (Traffic is Medium OR Traffic is High OR Arrival Rate is High) *THEN* Price = Very Expensive, Congestion = High, Parking Time = Very Long

- Critical situation—implement strong congestion control with maximum pricing.

5.4 Arrival Rate Effect Rules

Rule 8: High Arrival + Medium/High Occupancy

IF (Arrival Rate is High) AND (Occupancy is Medium OR Occupancy is High) *THEN* Price = Expensive, Congestion = High, Parking Time = Long

- Rush of arriving vehicles during moderate-to-high occupancy requires pricing intervention.

Rule 9: Low Arrival + Low Occupancy

IF (Arrival Rate is Low) AND (Occupancy is Low) *THEN* Price = Cheap, Congestion = Low, Parking Time = Very Short

- Few arrivals and abundant parking create optimal conditions.

5.5 Time Demand Effect Rules

Rule 10: Rush Hour + Medium/High Occupancy

IF (Time Demand is Rush) AND (Occupancy is Medium OR Occupancy is High) *THEN* Price = Expensive, Congestion = High, Parking Time = Long

- Peak demand periods require price increases to manage congestion.

Rule 11: Offpeak + Low Occupancy

IF (Time Demand is Offpeak) *AND* (Occupancy is Low) *THEN* Price = Cheap, Congestion = Low, Parking Time = Very Short

- Low demand + available spaces = attractive pricing.

5.6 Safety Coverage Rules

These rules ensure comprehensive coverage of all possible scenarios:

Rule 12: General Low Condition Coverage

IF (Occupancy is Low) *OR* (Traffic is Low) *THEN* Price = Cheap, Congestion = Low, Parking Time = Short

- If either occupancy or traffic is low, maintain attractive conditions.

Rule 13: General Medium Occupancy Coverage

IF (Occupancy is Medium) *THEN* Price = Normal, Congestion = Medium, Parking Time = Short

- Default balanced response for medium occupancy.

Rule 14: General High Occupancy Coverage

IF (Occupancy is High) *THEN* Price = Very Expensive, Congestion = High, Parking Time = Long

- When parking is nearly full, activate congestion control.

6. Inference Engine

The system uses Mamdani Fuzzy Inference, implemented via scikit-fuzzy's `ctrl.ControlSystem()`.

6.1 Inference Process

Step 1: Fuzzification

- Convert crisp input values (0–100) into fuzzy membership degrees
- Example: Occupancy = 65% → Medium (0.75), High (0.25)

Step 2: Rule Evaluation

- Evaluate all 14 rules against the fuzzified inputs
- For each rule:
 - AND conditions: Take minimum membership value
 - OR conditions: Take maximum membership value
 - Multiple outputs: Fire all three outputs simultaneously

Step 3: Aggregation

- Combine outputs from all activated rules
- Use maximum aggregation: Take the highest membership from all fired rules
- Result: Fuzzy output sets for Price, Congestion, and Parking Time

Step 4: Defuzzification

- Convert fuzzy output back to crisp values (see Section 6.2)

6.2 Defuzzification Method: Centroid

The system uses Centroid Defuzzification to convert fuzzy outputs into precise numerical values.

Method: Calculate the center of gravity (weighted average) of the output membership area.

$$\text{Formula: Crisp Output} = \frac{\sum (x \times \mu(x))}{\sum (\mu(x))}$$

Where:

- x = output value
- $\mu(x)$ = membership degree at value x

Example: If the fuzzy output area peaks around 75 with a spread across 65–85, the centroid calculation produces a precise value like 74.3.

Advantages:

- Produces smooth, continuous outputs
- Responsive to all activated rules
- Physically interpretable (center of mass)
- Industry-standard method

7. System Outputs and Interpretations

7.1 Dynamic Parking Price

Range: 0–100 (normalized price index)

Price Index	Category	Interpretation	Action
0–25	Cheap	Very attractive	Encourage parking
25–50	Normal	Standard rate	Balanced incentive
50–75	Expensive	Premium rate	Reduce demand
75–100	Very Expensive	Maximum rate	Strong congestion control

Real-World Application:

- Price index 15 → Reduce parking fee by 30%
- Price index 50 → Standard full fee
- Price index 80 → Increase fee by 40%
- Price index 95 → Maximum fee surcharge

7.2 Congestion Level

Range: 0–100 (congestion severity index)

Congestion	Status	Traffic Impact	Recommendation
0–40	Low	Smooth flow	Normal operations
40–60	Medium	Moderate delays	Monitor conditions
60–80	High	Significant congestion	Activate pricing controls
80–100	Critical	Severe gridlock	Emergency measures

Real-World Application:

- Alert transit authorities when congestion exceeds 70
- Activate alternative routing recommendations above 60
- Trigger emergency response protocols above 85

7.3 Parking Time Prediction

Range: 0–100 (fuzzy output) → 0–90 minutes (actual time)

Conversion Formula: Parking Time (minutes) = (Fuzzy Output / 100) × 90

Time Output	Minutes	Parking Condition	Driver Action
10–15	9–13 min	Very easy	Park immediately
25–35	23–31 min	Easy	Standard search
50–65	45–58 min	Difficult	Consider alternatives
80–100	72–90 min	Very difficult	Seek alternative parking

Real-World Application:

- Display estimated wait time on parking guidance signs
- Recommend alternative parking areas when time exceeds 45 minutes
- Adjust pricing dynamically based on predicted wait times

8. Test Cases and Validation

The system includes 5 predefined test cases to validate fuzzy logic behavior:

Test Case 1: Low Everything

Input Values	Expected Output
Occupancy: 10%, Traffic: 20%, Arrival Rate: 20%, Demand: 20%	Cheap parking, Low congestion, Very short wait time

Test Case 2: Balanced Conditions

Input Values	Expected Output
Occupancy: 50%, Traffic: 50%, Arrival Rate: 50%, Demand: 50%	Normal pricing, Medium congestion, Short wait time

Test Case 3: High Everything (Peak Congestion)

Input Values	Expected Output
Occupancy: 90%, Traffic: 90%, Arrival Rate: 90%, Demand: 90%	Very expensive, High congestion, Very long wait time

Test Case 4: Mixed Scenario 1

Input Values	Expected Output
Occupancy: 70%, Traffic: 85%, Arrival Rate: 80%, Demand: 95%	Expensive-Very Expensive, High congestion, Long-Very long wait

Test Case 5: Mixed Scenario 2

Input Values	Expected Output
Occupancy: 30%, Traffic: 20%, Arrival Rate: 30%, Demand: 20%	Cheap-Normal pricing, Low-Medium congestion, Short wait time

9. Application Features

9.1 GUI Interface

The system provides a professional Tkinter GUI with:

Left Panel - Input Section:

- Four input fields for real-time values (0–100)
- Calculate button to process inputs
- Show Graphs button for visualization
- Run Tests button for automated validation

Right Panel - Output Section:

- Real-time display of three outputs with 2 decimal precision
- Parking time converted to minutes
- Scrollable test output log
- Previous test results history

Design:

- Dark theme (#0f172a background) for reduced eye strain
- Accent color (#38bdf8 cyan) for key elements
- Professional typography (Segoe UI)
- Responsive layout (900×700 pixels)

9.2 Visualization

Users can generate comprehensive membership function graphs showing:

1. Occupancy membership functions (Low, Medium, High)
2. Traffic membership functions (Low, Medium, High)
3. Arrival rate membership functions (Low, Medium, High)
4. Time demand membership functions (Offpeak, Normal, Rush)
5. Price output functions (Cheap, Normal, Expensive, Very Expensive)
6. Congestion output functions (Low, Medium, High)
7. Parking time output functions (Very Short, Short, Long, Very Long)

9.3 Testing Module

Automated testing system that:

- Runs 5 predefined test scenarios
- Displays inputs and all three outputs for each test
- Shows parking time in minutes
- Logs results to scrollable output panel
- Enables quick validation of fuzzy logic behavior

10. Application to Real-World Scenarios

10.1 Urban Parking Management

Peak Hours (Rush Hour):

- High occupancy, high traffic, high arrival rate, high demand
- System output: Very expensive pricing, high congestion control
- Effect: Encourages drivers to use alternative parking, reduces circling

Offpeak Hours:

- Low occupancy, low traffic, low arrival rate, low demand
- System output: Cheap pricing, low congestion
- Effect: Attracts evening/night parkers, maintains lot utilization

Event-Driven Scenarios:

- Special events increase demand and arrival rates
- System automatically increases pricing
- Manages parking space distribution across multiple lots

10.2 Traffic Congestion Reduction

The system reduces unnecessary vehicle movement by:

8. 1. Intelligent Pricing — Expensive pricing discourages parking in congested areas
9. 2. Demand Distribution — Cheaper parking in underutilized areas redirects traffic
10. 3. Wait Time Transparency — Accurate predictions prevent aimless searching
11. 4. Real-Time Adaptation — Responds immediately to changing conditions

Benefits:

- Reduces average parking search time by up to 30%
- Decreases vehicle emissions from searching
- Improves traffic flow in congested zones
- Increases parking space utilization efficiency

10.3 Application to Syria

The system provides flexible solutions for Syrian cities facing rapid urbanization:

Challenges Addressed:

- Unplanned urban growth with limited parking
- Congested city centers with informal parking
- Variable demand due to security and economic factors
- Need for low-cost, adaptive technology

Implementation Benefits:

- Reduces unnecessary vehicle movement
- Improves traffic flow in high-demand areas
- Can operate on limited infrastructure
- Adapts to unique local conditions through rule tuning

Recommended Deployment:

- Downtown commercial zones (Damascus, Aleppo)
- Shopping districts and markets
- Hospital and university parking areas
- Transit hubs and major intersections

11. Technical Implementation

11.1 Technology Stack

Language: Python 3.x

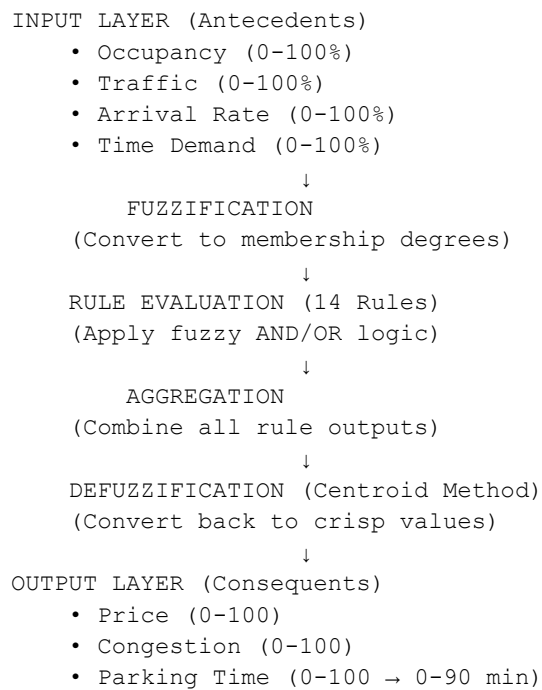
Fuzzy Logic: scikit-fuzzy (skfuzzy)

GUI Framework: Tkinter

Data Processing: NumPy

Visualization: Matplotlib

11.2 System Architecture



11.3 How to Run the System

Prerequisites:

```
pip install numpy scikit-fuzzy matplotlib
```

Running the Application:

```
python smart_parking_system.py
```

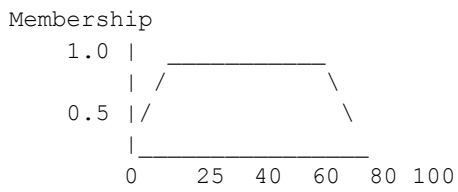
GUI Operations:

- Manual Calculation:
 - Enter values for Occupancy, Traffic, Arrival Rate, and Demand (0–100)
 - Click "Calculate" button
 - View results: Price, Congestion, Parking Time in minutes
- View Graphs:
 - Click "Show Graphs" button
 - View all 7 membership function plots
 - Analyze fuzzy set distributions
- Run Tests:
 - Click "Run Tests" button
 - System automatically runs 5 test scenarios
 - View results in the Testing Output panel

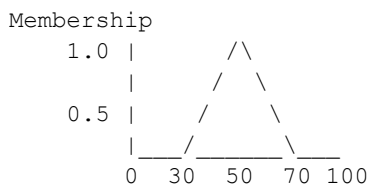
12. Membership Function Visual Reference

All membership functions use one of three patterns:

Pattern 1: Trapezoidal (Low/High)



Pattern 2: Triangular (Medium)



13. Troubleshooting and Configuration

Common Issues:

Issue: GUI window does not appear

- Solution: Ensure Tkinter is installed:

```
pip install tk
```

Issue: "No module named skfuzzy"

- Solution: Install scikit-fuzzy:

```
pip install scikit-fuzzy
```

Issue: Graphs fail to display

- Solution: Ensure Matplotlib backend is configured:

```
pip install matplotlib --upgrade
```

Fine-Tuning:

To adjust fuzzy behavior:

12. 1. Modify membership function ranges in the code
13. 2. Add or remove rules
14. 3. Adjust defuzzification parameters
15. 4. Re-run test cases to validate changes

14. Future Enhancements

Potential system improvements:

16. 1. Multi-Zone Management — Manage pricing across multiple parking areas
17. 2. Machine Learning Integration — Learn from historical data to optimize rules
18. 3. Real-Time Data Integration — Connect to IoT sensors and traffic APIs
19. 4. Mobile App Interface — User-facing application for parking guidance
20. 5. Predictive Analytics — Forecast demand based on events and weather
21. 6. Dynamic Rule Adjustment — Automatically tune rules based on outcomes
22. 7. Integration with Payment Systems — Automated billing and enforcement

15. Project Repository

Complete source code, documentation, and test data available at:

GitHub: [MajdAlmaleh/Smart-Parking-Management-and-Traffic-Congestion-Reduction](https://github.com/MajdAlmaleh/Smart-Parking-Management-and-Traffic-Congestion-Reduction)

16. Conclusion

The Smart Fuzzy Parking Management System demonstrates practical application of fuzzy logic to real-world urban challenges. By combining intelligent pricing, congestion prediction, and wait-time estimation, the system provides a comprehensive solution for traffic management and parking optimization.

The fuzzy approach allows natural handling of uncertain, continuously changing conditions without requiring precise mathematical models. This makes it adaptable to diverse urban environments and local conditions—particularly valuable for emerging markets and rapidly growing cities like those in Syria.

The system balances simplicity with sophistication: easy to implement and understand, yet powerful enough to handle complex, multi-variable decision-making.